

UNIVERSIDAD ADVENTISTA DE BOLIVIA

Carrera de Ingeniería de Sistemas

CONSULTORÍA PARA LA PROVISIÓN DE SOFTWARE PARA PYME

Materia: Ingeniería de Software (5to Semestre)

Docente: Msc. Maria Uriona

Equipo de Desarrollo:

Marializ Mamani

Leonardo Radek Condori Yucra, Juan Daniel Ancieta Toledo, Christian Fabian Gonzales Mamani

Fecha: 15 de junio 2026

ÍNDICE GENERAL

[Nota: Al descargar en Word, actualiza la Tabla de Contenido Automática aquí]

- **CAPÍTULO 1:** Introducción y Contexto

- CAPÍTULO 2: Ingeniería de Requerimientos
- CAPÍTULO 3: Diseño Arquitectónico y Base de Datos
- CAPÍTULO 4: Implementación y Prototipado Técnico
- CAPÍTULO 5: Gestión del Proyecto y Cultura DevOps
- CAPÍTULO 6: Calidad, Transferencia y Mantenimiento
- CAPÍTULO 7: Anexos y Trazabilidad

CAPÍTULO 1: Introducción y Contexto

El presente proyecto de ingeniería propone e implementa una solución de software orientada a la gestión financiera de Pequeñas y Medianas Empresas (PyMEs), diseñada específicamente para optimizar y modernizar el proceso de evaluación crediticia. El objetivo principal es dotar a los asesores y analistas de una herramienta multiplataforma robusta, segura y con capacidad de operación en entornos rurales sin conectividad (enfoque Offline-first), resolviendo las barreras geográficas habituales durante el levantamiento de datos en campo.

CAPÍTULO 2: Ingeniería de Requerimientos

2.1 Requerimientos Funcionales (Priorización MoSCoW)

MUST HAVE (Obligatorios):

- **RF-01 (Registro de Cliente):** El sistema debe permitir el registro del Expediente Digital validando CI/NIT únicos.
- **RF-02 (Volteo de Balances):** El sistema debe calcular y validar la ecuación contable ($\text{Activo} = \text{Pasivo} + \text{Patrimonio}$) con una tolerancia estricta de ± 10 Bs.
- **RF-03 (Evaluaciones Sectoriales):** Formularios dinámicos para rubros Agrícola, Pecuario y Producción.

SHOULD HAVE (Importantes):

- **RF-04 (Simulador de Pagos):** Generación de cronograma de amortización bajo el Sistema Francés.

2.2 Requerimientos No Funcionales, Seguridad y DevOps

- **RNF-01 (Offline-First):** La aplicación debe operar sin conexión a internet y sincronizar eventos posteriormente.
- **RNF-02 (Seguridad y Cifrado):** Los datos confidenciales deben cifrarse en AES-256 en reposo y viajar por TLS/SSL.
- **RNF-03 (Trazabilidad y Logs):** El sistema debe mantener logs de auditoría inmutables

usando Event Sourcing.

CAPÍTULO 3: Diseño Arquitectónico y Base de Datos

3.1 Arquitectura del Sistema

El núcleo backend se fundamenta en una **Arquitectura Hexagonal (Puertos y Adaptadores)** acoplada con **CQRS y Event Sourcing** empleando Axon Framework en Spring Boot (Java 17). Esto aísla la lógica de dominio. El frontend interactúa a través de adaptadores REST y está construido con **React y TypeScript (Vite)**.

3.2 Modelado de Base de Datos (MER)

[ALERTA ESTUDIANTE: Insertar aquí la imagen del Modelo Entidad-Relación (MER) de PostgreSQL exportado desde tu herramienta de modelado]

CAPÍTULO 4: Implementación y Prototipado Técnico

4.1 Diseño de Interfaces (Mockups)

[Insertar aquí las capturas de pantalla de tu frontend en React / Figma]

4.2 Diagramas UML

[Insertar imágenes de: Diagrama de Casos de Uso, Diagrama de Actividades y Diagrama de Estado]

CAPÍTULO 5: Gestión del Proyecto y Cultura DevOps

5.1 Cronograma de Trabajo

[Insertar aquí la imagen: 07_Cronograma.jpeg]

5.2 Flujo de Trabajo (GitFlow y CI/CD)

El equipo adoptó la estrategia GitFlow. Se mantiene una rama main protegida para código de producción y una rama develop para integración de características. Las implementaciones se

realizan mediante flujos automatizados de GitHub Actions que ejecutan linters y pruebas unitarias antes de permitir un Pull Request.

```
name: CI Backend Pipeline
on:
  push:
    branches: [ "main", "develop" ]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build with Maven
        run: ./mvnw clean verify
```

5.3 Definición de Listo (DoD)

- El código fuente ha sido subido al repositorio y no presenta conflictos.
- Las pruebas unitarias (JUnit 5) se ejecutan con éxito (>80% de cobertura en Controladores).
- La funcionalidad cumple estrictamente con los Criterios de Aceptación de la Historia de Usuario.
- El despliegue en el contenedor Docker local levanta sin errores de conexión a PostgreSQL.

CAPÍTULO 6: Calidad, Transferencia y Mantenimiento

6.1 Plan de Pruebas

Se estructuró una suite de QA sobre el adaptador web utilizando MockMvc y Mockito. Destaca la clase BalanceControllerTest, encargada de verificar la inmutabilidad de comandos y validar el rechazo automático de peticiones si el descuadre contable supera la tolerancia de ± 10 Bs.

6.2 Manual de Usuario

Se definió una guía operativa basada en roles. El **Asesor de Campo** gestiona la creación del expediente único y la verificación del checklist de documentos físicos. El **Analista Financiero**

opera los módulos de Volteo de Balances y Evaluaciones Sectoriales, con indicadores semaforizados para evaluar rentabilidad.

6.3 Manual Técnico

Para el mantenimiento y despliegue local, el entorno utiliza H2 Database. Para el pase a producción, se empaqueta la aplicación Java en un Dockerfile multi-etapa y se inyectan variables de entorno para la conexión segura al clúster de PostgreSQL (jdbc:postgresql://HOST/bdp_credit_db).

6.4 Plan de Capacitación

El cronograma formativo contempla un Taller Funcional para el área de negocios del BDP y una Transferencia Tecnológica para el área de TI (Administradores), enfocada en el monitoreo de Axon Server y gestión de contenedores.

CAPÍTULO 7: Anexos y Trazabilidad

Anexo A - Matriz de Trazabilidad

Req. ID	Descripción	Módulo Code	Test Unitario
RF-02	Validación ± 10 Bs en Ecuación	BalanceController	validarBalanceFinal_ retornara...
RF-04	Sistema Amortización Francés	SimuladorService	Pendiente